Celerity

The money is already committed. Celerity makes the last mile instant.

Programmable disaster disbursement for farmers — one trigger, many funders, instant cash-out.

Rise In × Stellar APAC Hackathon 2026 · **Local Finance & Real World Access**

Track: Local Finance & Real World Access · Deadline: July 15, 2026 · Ethan Dreiz Baltazar · Pamantasan ng Lungsod ng Maynila · START-DOST / GDG on Campus PLM

---

1. Executive Summary

Celerity is a programmable disbursement rail on Stellar for disaster relief. Funders — a national crop insurer, a regional fund like the ADB's Asia Pacific Disaster Response Fund, a private foundation — deposit money into a shared on-chain escrow, each with its own earmark and its own release rule. When an objective weather trigger fires (a PAGASA typhoon signal at or above a set threshold for a given region, cross-checked against Japan's RSMC Tokyo), the smart contract releases payouts automatically to pre-registered farmers in the affected area, converts them to spendable pesos through a Stellar anchor, and logs every movement so each funder can see exactly where its money went. No agency hand-off, no physical check distribution, no waiting.

What makes Celerity different is that it is **not another crop insurer** and **not a faster claims processor** — it is the settlement layer *underneath* them. The Philippine Crop Insurance Corporation is already piloting parametric (trigger-based) crop insurance, which proves the model. But PCIC is single-funder, peso-only, and still pays by physically distributing indemnity checks at regional offices. Celerity is the rail that lets *international and private funders co-fund the same trigger* and delivers the payout as instant digital cash — the multi-funder, cross-border, check-free layer PCIC's own parametric program will need to scale.

One-liner

A disaster-relief disbursement rail where an objective weather trigger releases pooled funds from many independent funders at once, enforced via a Soroban smart contract, cashing out to local currency through Stellar anchors.

Pillars Table

Pillar	What it does	Status
Shared escrow pool	Holds many funders' deposits, each earmarked with its own rule	Build (Week 1)
Oracle trigger	Releases on a signed weather-signal condition (PAGASA / JMA)	Build (Week 1)
Multi-funder release	One trigger fires every funder's sub-pool independently	Build (Week 1)
Recurring payout	Pays in installments (e.g. ₱50k/mo × 2) not one lump	Build (Week 2)
Farmer registry	Pre-enrolled beneficiary list, keyed to region	Build (Week 2)
Anchor cash-out	Converts USD/stablecoin → PHP for the farmer	Stub / SEP-31 mock (Week 2)
Transparent ledger	Per-funder record of every release	Build (Week 2)
Live oracle feed	Real PAGASA/JMA data ingestion	Stub (Roadmap)

2. Hackathon Submission Fields

2.1 Track

Selected Track: Local Finance & Real World Access.

Celerity is a clean fit because it is infrastructure for *real-world money reaching real people* — pre-registered farmers receiving disaster relief as spendable local currency, not a token they can't use. The anchor cash-out (SEP-31/SEP-6) is the core of the "real world access" claim: the value lands as pesos through the same licensed rails Filipinos already use. And it is "local finance" in the literal sense — it routes international disaster capital (USD) into a local agricultural economy (PHP) that today receives that money weeks later through layers of agencies.

2.2 Problem Statement

When a typhoon destroys a farmer's crop, the money to help them recover usually *already exists* — in a national crop-insurance fund, a regional disaster pool, or an NGO earmark. What's slow and lossy is the last mile. The Philippine Crop Insurance Corporation processes most claims within about 20 working days and is targeting 10, but the payout itself is still

distributed as **physical indemnity checks handed out at regional offices** — the search record shows distributions spanning weeks (e.g. Jan 5–26 at one regional office). For an uninsured farmer, or one not registered under the RSBSA system, there is no fast path at all. And when a *foreign* funder (ADB's APDRF, an overseas foundation) wants to help, its money cannot reach a farmer directly — it routes through national then local agencies, often via delivery partners, taking additional weeks.

Existing solutions don't close this because they each reintroduce an intermediary. PCIC's parametric pilot removes the *field inspection* but still ends in agency-run check distribution and is single-funder and peso-only — it structurally cannot ingest foreign USD and route it to a farmer, nor let that foreign funder verify where its money went. The missing mechanism is a neutral settlement layer that (a) lets multiple independent funders act on one objective trigger and (b) delivers value directly to the beneficiary as spendable local cash.

2.3 Proposed Solution

Funders deposit into a shared Soroban escrow, each deposit earmarked as a sub-pool with its own payout rule and recipient scope. An oracle submits a signed weather event (region, typhoon signal level). The contract checks each sub-pool's condition against that event; for every sub-pool whose condition is met, it releases the payout to the registered farmers in that region, on the funder's chosen schedule, and emits a per-funder ledger event. Payouts convert to PHP through a Stellar anchor at the edge.

Supporting features layered on the core:

- **Multi-funder sub-pools** — independent earmarks, one shared trigger.
- **Recurring installments** — staged payout (e.g. ₱50k/mo × 2) instead of a single lump, matched to rebuild needs.
- **Farmer registry** — pre-enrolled beneficiaries keyed to region, mirroring RSBSA enrollment.
- **Oracle-signed trigger** — event data signed by an authorized key (PAGASA-role), so the contract verifies a signature, not a document.
- **Anchor cash-out** — USD/stablecoin → PHP via SEP-31, so the farmer gets spendable money.
- **Transparent per-funder ledger** — every release attributable to its funder.

Core claim

Celerity is not a crop insurer — it is the settlement rail that lets a national insurer, a regional fund, and a foreign foundation co-fund the same typhoon trigger and pay a farmer instant, spendable pesos, without a single agency hand-off.

2.4 Target Users / Audience

- **Disaster funders** — PCIC, ADB's APDRF, the national/local calamity funds (NDRRMF), NGOs — anyone holding committed disaster money that must reach

individuals fast.

- **Pre-registered farmers** — RSBSA-enrolled (or co-op/LGU-enrolled) smallholders in typhoon-exposed regions; the underserved, often unbanked, non-technical beneficiary.
- **Regional/international funders needing accountability** — foreign funders who must prove to their own stakeholders that money landed where intended.
- **Cooperatives and LGUs** — who maintain the beneficiary registry and act as the local enrollment layer.

2.5 Expected Stellar Integration

- **Soroban smart contracts (Rust)** — escrow, trigger logic, multi-funder release.
- **Stellar accounts + multisig** — scoping funder vs. oracle vs. admin authority.
- **Anchors (SEP-31 / SEP-6/24)** — fiat cash-out USD → PHP at the edge.
- **Stellar Testnet** — deploy + demo target.
- **Native asset/path payment** — USDC/PHPC handling and conversion at settlement.

2.6 Why Stellar Specifically

- **Anchor network** — a live fiat off-ramp into this exact market (Coins.ph-style PHPC rails) that a generic chain would have to hand-roll; this is the "real world access" the farmer actually needs.
- **Sub-cent, seconds-fast settlement** — makes instant, direct-to-farmer payout demonstrable live, versus percentage-fee, multi-day bank rails.
- **Native multisig/account primitives** — cleanly scope who can fund, who can sign a weather event, and who can administer — without hand-built access control.

Why not a generic EVM chain?

On Stellar the fiat cash-out is a native anchor call; on a generic chain the farmer is left holding a token they can't spend, and you rebuild the entire off-ramp yourself.

---

3. Competitive Positioning

3.1 The Competitor: PCIC Parametric Crop Insurance

- **Who they are:** The Philippine Crop Insurance Corporation, the government crop insurer — real, funded (₱4.5B+ national budget, 4.2M farmers covered in 2024), and actively rolling out parametric insurance (announced July 2025, piloting in the wet harvest season).
- **Their approach:** Replace time-consuming field inspection with remote sensing — measure wind velocity via satellite/weather data, pay insured RSBSA farmers, targeting a 20→10 day claims cycle.

- **The dependency Celerity doesn't have:** PCIC is single-funder (government), peso-only, and its last mile is still **physical check distribution through regional offices**. It cannot ingest foreign USD disaster capital, cannot let an external funder verify disbursement, and does not settle as instant spendable digital cash.

3.2 The Armor (say this to judges verbatim)

PCIC going parametric doesn't threaten Celerity — it validates it. They proved the trigger works. But PCIC is one funder, one currency, and still pays by handing out checks at a regional office. Celerity is the rail underneath: it lets ADB, an NGO, and PCIC itself co-fund one typhoon trigger and pay a farmer instant pesos in any currency, with every funder able to see where its money went. We are the settlement layer their own parametric program will need to go multi-funder and check-free.

Comparison Table

Dimension	PCIC Parametric	Celerity
Core mechanism	Weather trigger → insurer pays insured farmers	Weather trigger → many funders' sub-pools pay registered farmers
Funders supported	Single (government)	Many, independent, earmarked
Currency	PHP only	USD / stablecoin / PHPC → PHP at edge
Cross-border capital	No	Yes (ADB APDRF, foreign NGOs)
Last mile	Physical checks at regional office	Instant digital cash-out via anchor
Funder-level transparency	Opaque to outside funders	Per-funder on-chain ledger
Trust model	Institutional	Signed oracle event + registry

---

4. Feature Breakdown

4.1 Shared Escrow Pool — many funders, one contract

Each funder calls `deposit` and creates a sub-pool: an earmarked balance plus a rule (region, signal threshold, payout amount, schedule). Balances never commingle; the contract tracks each funder's remaining balance and terms independently.

- Each sub-pool is fully independent — one funder's release or exhaustion never touches another's.
- Funders can top up or withdraw their *own* unspent balance; never another's.

- A single farmer can be covered by multiple sub-pools (e.g. PCIC tuition-equivalent + NGO living support) and receive from each on the same trigger.

> **Scope boundary** Build the multi-sub-pool data model and *one* funded sub-pool in the live demo. Show a second funder's sub-pool as pre-seeded state (not funded live on stage) to prove independence without spending demo time on two funding flows.

4.2 Oracle Trigger — a signed event, not a read document

An authorized oracle key submits `report_event(region, signal_level)` with a signature. The contract verifies the signature against a stored PAGASA-role public key and treats the event as ground truth. It does **not** read or interpret any document — it checks a signature and a number.

- The contract compares `signal_level` to each sub-pool's threshold and `region` to its scope.
- Backup oracle (JMA/RSMC Tokyo role key) can be added as a second authorized signer for cross-check.

> **Oracle note** The trust of the whole system rests on the oracle key; a compromised key could release funds on a false event. Mitigation: time-boxed/scoped oracle keys plus funder-side spot-check sampling on released events (see §5.1). In the demo the oracle is a Node.js signer simulating PAGASA; live-feed ingestion is roadmap (§4.8).

4.3 Multi-Funder Release — one trigger, all act

On a valid event, the contract iterates matching sub-pools and releases each independently, emitting one ledger event per funder per farmer.

- Release is idempotent per (event, farmer, sub-pool) — a re-submitted event cannot double-pay (see §5.2).
- If a sub-pool has insufficient balance, it flags rather than partially failing the whole event.

4.4 Recurring Payout — staged, not lump

A sub-pool can specify installments; the contract schedules `next_release` per farmer and releases on cadence after the trigger.

- Pull-based: recipient (or their co-op) calls `claim` when an installment is due, avoiding a backend scheduler.
- Funder can pause/adjust a farmer's remaining installments (exception authority only).

4.5 Farmer Registry — pre-enrolled, region-keyed

`register_farmer(address, region)` maintained by an LGU/co-op admin role, mirroring RSBSA enrollment. The contract pays only registered addresses in the triggered region.

- The registry is the human trust edge — the contract doesn't decide who is a farmer, it pays the verified list (parallel to how RSBSA registration already gates PCIC coverage).

4.6 Anchor Cash-out — spendable pesos at the edge

Released value routes to a SEP-31 anchor for USD/stablecoin → PHP conversion.

> **Scope boundary** Celerity is not a licensed anchor and won't be by July 15. Build the on-chain release fully; mock the anchor leg with a stubbed SEP-31 receiver and narrate the real integration point. Everything on-chain is real; the fiat conversion is the honest stub.

4.7 Transparent Ledger — per-funder receipts

Every release emits an event carrying (funder, farmer, amount, event_id, timestamp). A funder view filters to its own releases.

4.8 Stub Features — Roadmap (honest scope)

Stubs are deliberate scope boundaries, not shortfalls.

Feature	Demo Treatment	Why It's Roadmap
Live PAGASA/JMA feed	Node.js signer simulates a signed event	Real feed needs authorized data agreements + infra beyond hackathon scope
Licensed anchor cash-out	Stubbed SEP-31 receiver, narrated	Requires a licensed VASP partner (regulatory, not technical)
Second funder funded live	Pre-seeded sub-pool shown as state	Two live funding flows waste demo time; independence is provable from state
RSBSA identity integration	Local registry map	Government identity integration is a partnership, not a weekend build

---

5. Critical Technical Gotchas

5.1 Oracle Key Compromise

> **Risk** The oracle key is a single point of trust. If it's stolen or misissued, an attacker submits a false "signal 5, region X" event and drains real sub-pools faster than any paper process could catch — the very speed that is the product becomes the attack surface.

Fix: scope and time-box the oracle key, require a second signer (JMA-role) above a payout size threshold, and gate large releases behind a short challenge window with funder spot-

check sampling.

```rust
// require dual-sig above a threshold; single-sig only for small, fast payouts
fn report_event(e: Env, region: u32, signal: u32, sigs: Vec<Signature>) {
    let required = if payout_exposure(&e, region) > DUAL_SIG_LIMIT { 2 } else {
    verify_authorized_signers(&e, &sigs, required); // PAGASA + optional JMA ro
    // ... proceed to release
}
```

5.2 Double-Release / Replay

> **Risk** The same event is submitted twice (network retry, or a malicious replay), and a farmer is paid twice for one typhoon, draining sub-pools.

Fix: make release idempotent on a composite key and record settled tuples.

```rust
let k = (event_id, farmer.clone(), pool_id);
if e.storage().persistent().has(&k) { continue; } // already settled
e.storage().persistent().set(&k, &true);
// ... transfer
```

5.3 Flag-not-Fail on Insufficient Balance

> **Risk** A sub-pool runs dry mid-event; a naive loop reverts the whole transaction and *no* farmer from *any* funder gets paid.

Fix: isolate per-sub-pool failure — flag the underfunded pool, continue releasing the solvent ones, emit a `PoolExhausted` event for funder top-up.

5.4 Region/Threshold Mismatch Across Funders

> **Risk** Funders define regions or signal thresholds inconsistently, so one funder pays and another silently doesn't for the same farmer — looking like a bug or unfairness.

Fix: enforce a shared region enum and signal scale at the contract boundary; reject sub-pool creation that doesn't conform, so all funders speak one vocabulary.

---

6. Technical Architecture

6.1 System Overview

Celerity is a Soroban contract (escrow, trigger, multi-funder release) deployed to Stellar Testnet, driven by a React frontend with a funder view and a farmer/claim view, fed by a

Node.js off-chain oracle signer that simulates PAGASA/JMA weather events, and settling through a stubbed SEP-31 anchor for PHP cash-out.

Layer	Technology	Role
Smart Contract	Soroban (Rust)	Escrow, oracle-triggered multi-funder release, registry, ledger
Frontend	React + Stellar SDK	Funder setup, farmer claim, live ledger view
Auth	Stellar accounts + multisig	Scope funder / oracle / registry-admin roles
Off-chain	Node.js oracle signer	Simulates signed PAGASA/JMA events
Anchor	Stubbed SEP-31 receiver	USD/stablecoin → PHP cash-out (mock)
Deploy target	Stellar Testnet	Demo + submission

6.2 Core Storage Schema

```rust
```

```rust
#[contracttype]
pub struct SubPool {
    pub funder: Address,
    pub balance: i128,
    pub region: u32,
    pub signal_threshold: u32,
    pub payout_per_farmer: i128,
    pub installments: u32,      // 1 = lump, >1 = recurring
    pub status: PoolStatus,     // Active | Paused | Exhausted
}

#[contracttype]
pub struct Farmer {
    pub addr: Address,
    pub region: u32,
    pub registered_by: Address, // LGU/co-op admin
}

#[contracttype]
pub enum DataKey {
    Pool(u64),                  // pool_id -> SubPool
    FarmerReg(Address),         // addr -> Farmer
    Settled(u64, Address, u64), // (event_id, farmer, pool_id) -> bool
    OracleKey,
    Admin,
}
```

6.3 Full Contract Function Set

rust

```
// --- funder ---
fn deposit(e: Env, funder: Address, region: u32, threshold: u32,
           payout: i128, installments: u32) -> u64; // creates sub-pool
fn top_up(e: Env, pool_id: u64, amount: i128);
fn withdraw_unspent(e: Env, pool_id: u64);
fn pause_pool(e: Env, pool_id: u64);

// --- registry (LGU/co-op admin) ---
fn register_farmer(e: Env, addr: Address, region: u32);
fn remove_farmer(e: Env, addr: Address);

// --- oracle ---
fn report_event(e: Env, region: u32, signal: u32, sigs: Vec<Signature>) -> u64;

// --- release / claim ---
fn settle_event(e: Env, event_id: u64);        // releases all matching pools
fn claim(e: Env, farmer: Address, pool_id: u64); // pull next installment

// --- views ---
fn pool(e: Env, pool_id: u64) -> SubPool;
fn funder_ledger(e: Env, funder: Address) -> Vec<Release>;
```

6.4 Fallback: Manual Signed Event

If SEP-31 anchor mocking or full weather-signing isn't ready, fall back to an admin-signed `report_event` triggered by a button in the demo UI — the release logic (the actual innovation) is unchanged; only the event *source* is manual. Switch to this by Week 3 Wednesday if the Node signer slips.

---

7. Demo Flow — Stage Script

Target runtime: **3.5 minutes**, live on Testnet with a backup recording.

Step 1 — The setup (25s)

Open on the funder view. Show two sub-pools already seeded: "ADB APDRF — USD" and "NGO — stablecoin," both scoped to Region V (Bicol), threshold signal ⩾ 3. State: "Two independent funders, one region, money locked and waiting."

Step 2 — Fund a pool live (20s)

As a third funder, deposit into a new sub-pool live — show the balance lock and the earmark. "Each funder keeps its own money and its own rule."

Step 3 — Register a farmer (15s)

As the LGU admin, register a farmer in Region V. "The co-op enrolls the farmer — the contract doesn't guess who's eligible, it pays the verified list."

Step 4 — The typhoon hits (30s)

Run the Node oracle signer: sign "Region V, signal 4." Submit `report_event`. This is the hero beat — an objective weather signal, no human judgment, entering the contract.

Step 5 — Instant multi-funder release (40s)

Call `settle_event`. All three sub-pools release to the farmer at once, in seconds. Show the balances drop and three separate ledger entries appear. "One typhoon signal. Three funders paid the same farmer. No agency, no check."

Step 6 — Anchor cash-out (30s)

Show the released value routing through the stubbed SEP-31 anchor to a PHP balance. "USD and stablecoin become spendable pesos at the edge — the part PCIC still does with a physical check weeks later."

Step 7 — The mic-drop (30s)

Show the recurring installment: "And it's not a lump — ₱50k this month, ₱50k next, matched to rebuild." Then the ledger view filtered per funder.

> **Close the demo with this** *PCIC is going parametric — they proved the trigger. But they're one funder, one currency, and they still hand out checks at a regional office. I just paid one farmer from three funders, in two currencies, as instant pesos, live. Celerity is the rail their own program will need.*

---

8. Build Timeline — Now to July 15 (AI-Leveraged)

This is a 7-day sprint, not 3 weeks. The plan assumes heavy use of a frontier coding model (Claude Fable 5 / Opus 4.8 via Claude Code) to parallelize what a solo builder normally serializes. The rule: **AI drafts, you verify on-chain.** No step is "done" until it compiles, tests pass, and it's deployed to Testnet — the model accelerates drafting, it does not replace verification.

How to actually use the AI (this is the leverage)

- **Contract-first, AI-drafted, human-verified.** Give the model §6.2/§6.3/§10 as the spec and have it generate the full Soroban contract in one pass, then you compile and fix. Frontier models are strong at Rust/Soroban scaffolding; the win is skipping the blank-page phase.
- **Make the AI write the tests, especially the nasty ones.** The two bugs that lose this hackathon are double-release (§5.2) and dry-pool revert (§5.3). Have the model

generate adversarial test cases for both *before* you trust the release loop. This is where AI assistance pays off most — exhaustive edge-case enumeration is exactly what it's good at.

- **Parallelize contract and frontend.** Use one Claude Code session on the Rust contract and a second on the React frontend against a mocked contract interface, so the two tracks advance simultaneously instead of frontend waiting on contract.
- **Use AI for the oracle signer boilerplate.** The Node.js Ed25519 signing utility is standard code — generate it, don't write it by hand.
- **Caveat:** don't architect around a specific model being available (safeguards may route some prompts to Opus 4.8); and never paste a private key into any tool. Treat AI as a fast junior engineer whose every PR you review.

Days 1–3 — The Core (July 8–10)

- AI-draft the full Soroban contract from §10; you compile, fix, and get `cargo build` green.
- Implement + AI-test `deposit`, `register_farmer`, `report_event` (sig verify), `settle_event`.
- Idempotency (§5.2) and flag-not-fail (§5.3) built in from the first commit, with AI-generated adversarial tests.
- Deploy to Testnet by end of Day 3. **Milestone: a signed event releases one pool to one farmer on-chain.**

Days 4–5 — The Product (July 11–12)

- React funder view (deposit, ledger) + farmer/claim view, developed in parallel from Day 1 against the mock.
- Node.js oracle signer (AI-generated) wired to the live contract.
- Recurring installment `claim`; SEP-31 anchor cash-out stub.
- **Milestone: full end-to-end flow works on Testnet — deposit → event → multi-pool release → cash-out stub.**

Days 6–7 — Polish + Submission (July 13–15)

- Seed the multi-funder demo state; rehearse the 3.5-min script to muscle memory.
- Record the backup demo video (do this Day 6, not Day 7 — never leave the recording to the deadline).
- AI-draft the submission-form fields from §2; you edit for voice.
- Final Testnet redeploy; cut-line review.

The opportunity the extra AI capacity buys you

Because drafting is faster, you have headroom for **one** ambitious stretch item — pick at most one, only if Days 1–5 milestones hit on time:

- A second funder funded *live* on stage (not pre-seeded), making the multi-funder claim undeniable.
- A minimal *real* oracle read — pull an actual PAGASA/JMA signal number from a public source and sign it, instead of a hardcoded value.
- A per-funder ledger visualization polished enough to screenshot for the pitch deck.

Cut Line (if behind)

> **Safe to cut** Cut: live funder deposit (pre-seed all funders), recurring installments (show lump only), second oracle signer, all stretch items. Win condition: one signed weather event releases two pre-seeded funders' money to one registered farmer, live on Testnet, with a per-funder ledger. That alone proves the core claim.

---

9. Risks & Mitigations

Risk	Likelihood	Mitigation
Oracle key compromise (§5.1)	Med	Scoped/time-boxed keys, dual-sig above threshold, spot-check sampling
Double-release/replay (§5.2)	High	Idempotent settled-tuple keys
Anchor cash-out not ready	High	Stubbed SEP-31 receiver, narrated as integration point
Node oracle signer slips	Med	Fallback to admin-signed button event (§6.4)
"Isn't this just PCIC?" (Q&A)	Certain	§3.2 armor: PCIC validates the trigger; Celerity is multi-funder, cross-border, check-free rail
"Farmers aren't waiting months"	Certain	Concede it — pitch check-free instant cash-out + foreign co-funding, not raw speed
Testnet resource/downtime	Low	Deploy early Week 1, keep a redeploy script
Demo-day failure	Med	Backup recording of the full 3.5-min flow

---

10. Appendix — Contract Skeleton (Rust / Soroban)

```rust
```

```rust
#![no_std]
use soroban_sdk::{contract, contractimpl, contracttype, Address, Env, Vec, Byte

#[contracttype]
pub enum PoolStatus { Active, Paused, Exhausted }

#[contracttype]
pub struct SubPool {
    pub funder: Address, pub balance: i128, pub region: u32,
    pub signal_threshold: u32, pub payout_per_farmer: i128,
    pub installments: u32, pub status: PoolStatus,
}

#[contracttype]
pub struct Farmer { pub addr: Address, pub region: u32, pub registered_by: Addr

#[contracttype]
pub enum DataKey {
    Pool(u64), FarmerReg(Address), Settled(u64, Address, u64),
    OracleKey, Admin, NextPoolId, NextEventId,
}

#[contract]
pub struct Celerity;

#[contractimpl]
impl Celerity {
    pub fn init(e: Env, admin: Address, oracle: BytesN<32>) { /* set Admin, Ora

    pub fn deposit(e: Env, funder: Address, region: u32, threshold: u32,
                   payout: i128, installments: u32) -> u64 {
        funder.require_auth();
        // create SubPool, transfer funds into contract, return pool_id
        unimplemented!()
    }

    pub fn top_up(e: Env, pool_id: u64, amount: i128) { unimplemented!() }
    pub fn withdraw_unspent(e: Env, pool_id: u64) { /* funder-auth only */ unim
    pub fn pause_pool(e: Env, pool_id: u64) { /* funder-auth only */ unimplemen

    pub fn register_farmer(e: Env, addr: Address, region: u32) {
        // registry-admin auth; store Farmer
        unimplemented!()
    }
    pub fn remove_farmer(e: Env, addr: Address) { unimplemented!() }
```

```
    pub fn report_event(e: Env, region: u32, signal: u32, sig: BytesN<64>) -> u
        // verify sig against OracleKey; store event; return event_id
        unimplemented!()
    }

    pub fn settle_event(e: Env, event_id: u64) {
        // for each Active pool matching region & signal >= threshold:
        //   for each registered farmer in region:
        //     k = Settled(event_id, farmer, pool_id); skip if set
        //     transfer payout (or first installment); mark settled; emit Relea
        //   flag PoolExhausted instead of reverting on low balance
        unimplemented!()
    }

    pub fn claim(e: Env, farmer: Address, pool_id: u64) {
        // pull next installment if due; advance schedule
        unimplemented!()
    }

    pub fn pool(e: Env, pool_id: u64) -> SubPool { unimplemented!() }
    // pub fn funder_ledger(...) -> Vec<Release> { ... }
}
```

---

11. Scalability — APAC Implementation Plan

Celerity's engine is disaster- and country-agnostic: escrow, a signed trigger, multi-funder release, anchor cash-out. The contract does not change per country — only three things swap: the **oracle** (which weather authority signs), the **anchor** (which currency it cashes out to), and the **registry** (which national system enrolls farmers). That's why expansion is a configuration, not a rewrite.

Why APAC needs a shared rail (the fragmentation is the opportunity)

Crop insurance across the region is a patchwork, not a system. The Philippines, Thailand, and Vietnam have long-running schemes; Indonesia, the Philippines, and Thailand have scaled subsidized crop insurance for rice and maize; India runs weather-index insurance at massive scale with government subsidy. Indonesia's AUTP alone had insured ~5.26M hectares and paid over IDR 6.3 trillion (~$360M) by 2022, and Indonesia has piloted satellite/weather-index parametric products. Meanwhile Lao PDR and Malaysia have *no* agricultural insurance yet (governments want to start), and ADB is actively promoting weather-based index insurance region-wide.

The consequence: every country has a **different insurer, different currency, different trigger design, and no shared way for cross-border disaster capital to reach a farmer directly.** A regional fund like ADB's APDRF that wants to help farmers in three countries

after one typhoon season must route through three different national systems. Celerity is the neutral settlement layer that lets one funder act across all of them.

Phased rollout

Phase	Scope	Oracle	Anchor / Currency	Registry
0 (hackathon)	PH rice farmers, one region	PAGASA (mock) + JMA backup	PHPC / USDC → PHP (stub)	RSBSA-style list
1	PH, live	PAGASA feed	Licensed PH anchor (Coins.ph-class)	RSBSA via LGU/co-op
2	+ Indonesia	BMKG	IDR anchor	AUTP registry
3	+ Vietnam, Thailand	VNMHA / TMD	VND, THB anchors	national schemes
4	Regional fund layer	multi-source	multi-currency pool	cross-scheme

What is genuinely reusable vs. what is per-country work

- **Reusable (zero change):** the Soroban contract, the multi-funder sub-pool model, the release/idempotency logic, the ledger, the frontend.
- **Per-country config:** oracle public key + region enum, anchor endpoint + asset, registry source.
- **Per-country *hard* work (be honest):** each new market needs a licensed anchor partner and regulatory fit, and integration with that country's farmer registry. This is a partnership/compliance effort, not a code effort — which is exactly why the pitch is "win one corridor, then each new country is a plug-in, not a rebuild," not "we serve all of APAC on day one."

The regional end-state

A single pool where ADB (USD), a national insurer (local currency), and an NGO (stablecoin) all co-fund one weather trigger, and a farmer in the Philippines, Indonesia, or Vietnam is paid in their own currency the day the storm hits — each funder seeing exactly where its money went. No existing system does this because no existing system is currency-neutral, funder-neutral, and settlement-native at once. That combination is only possible on a rail like Stellar.

---

12. Anticipated Q&A — Judge Questions & How to Answer

Each answer is written to be said out loud in 15–25 seconds. The pattern throughout:

concede the true part, then pivot to the wedge.

Q1. "Isn't this just PCIC? They're already doing parametric."

Concede it, then reframe: *"PCIC going parametric validates the trigger — that's why I'm confident it works. But PCIC is one funder, peso-only, and still pays by handing out physical checks at regional offices. Celerity is the rail underneath: it lets ADB, an NGO, and PCIC co-fund the same trigger and pay a farmer instant digital pesos in any currency. I'm not their competitor — I'm the multi-funder, cross-border layer their own program will need to scale."*

Q2. "Farmers only wait ~20 days, and PCIC is cutting it to 10. Where's the pain?"

"You're right that insured, RSBSA-registered farmers get paid in about 20 days — I'm not claiming months. My wedge isn't raw speed against PCIC. It's three things PCIC structurally can't do: ingest foreign USD disaster money, deliver it as instant spendable cash instead of a check collected weeks later, and let that foreign funder verify where it went. And for the uninsured or unregistered farmer, there's no fast path today at all."

Q3. "Why blockchain at all? A government database with a button does this."

"For a single national insurer in one currency — you're right, a database works. The moment you have multiple independent funders, in different currencies, who don't trust each other's books and need cross-border settlement into spendable local cash — that's exactly what a shared ledger plus anchors does natively and a database can't. The chain earns its place on multi-funder trust and fiat settlement, not on the trigger."

Q4. "Who is the oracle in production, and why should anyone trust it?"

"PAGASA — the Philippine weather authority, under DOST — issues the official typhoon signals, cross-checked against Japan's RSMC Tokyo, which is the regional standard for the whole Western Pacific. The contract verifies a signature from an authorized weather-authority key; it never reads or interprets a document. And above a payout threshold I require dual-signature, so no single compromised key can drain a pool."

Q5. "What stops a stolen oracle key from draining every pool?"

"That's the real risk, and I've designed for it. Oracle keys are scoped and time-boxed; releases above a threshold need a second signer; and large releases pass through a short challenge window with funder-side spot-check sampling. The speed is gated exactly where the money is largest."

Q6. "You're not a licensed money operator. Isn't the cash-out fake?"

"Correct — I'm not a licensed anchor and I'm not pretending to be. Everything on-chain is real and live on Testnet. The fiat cash-out is where a licensed anchor like Coins.ph plugs in — I mock that leg and I'm explicit about it. That's the architecture working as designed: I build the programmable release layer, the anchor does the regulated conversion."

Q7. "Two funders paying the same farmer — isn't that double-dipping?"
"Only if they're funding the same thing. The design assumes distinct purposes — e.g. a national insurer covers crop loss while an NGO covers living support — each with its own earmark and rule. The contract keeps them independent and logged separately, so it's transparent, not duplicative. If two funders genuinely overlap, the ledger makes that visible for them to resolve."

Q8. "How is this APAC, not just Philippines?"

"The engine is country-agnostic — only the oracle, anchor, and registry swap per country. The region is a patchwork: Indonesia, Thailand, Vietnam, and India each have different insurers and currencies, and Laos and Malaysia have none. A regional fund wanting to help across borders has no shared rail today. Celerity is that rail — win the PH corridor, then each new country is a config, not a rebuild."

Q9. "What's actually built vs. stubbed?"

"Built and live on Testnet: the full escrow, multi-funder release, signed-trigger logic, registry, and per-funder ledger. Stubbed and clearly labeled: the licensed anchor cash-out, and the live weather feed. I stubbed exactly the two things that are partnership or infrastructure problems, not engineering ones — and I'll tell you that on stage before you ask."

Q10. "If you win the funding, does this scale into a real business?"

"Yes — the buyers are institutional: national insurers, ADB-class regional funds, and NGOs who need auditable cross-border disbursement. The wedge is the pre-registered-beneficiary rail, and the same engine extends from disaster relief to survivor benefits, stipends, and payroll. One rail, many objective conditions."

Celerity · *Disaster money that moves itself*

Ethan Dreiz Baltazar · PLM · START-DOST CTO / GDG on Campus PLM · Rise In × Stellar APAC Hackathon 2026 Celerity © 2026 · Ethan Dreiz Baltazar · PLM · p.final